University of Windsor

05 Introduction to Distributed Database Management System

Kalyani Selvarajah
School of Computer Science
University of Windsor

Advanced Database Topics
COMP 8157 01
Winter 2026

Today's Agenda

System Architectures

Introduction to Distributed Databases

Advantages and Disadvantages of DDBMSs

Architectures of a DDBMS

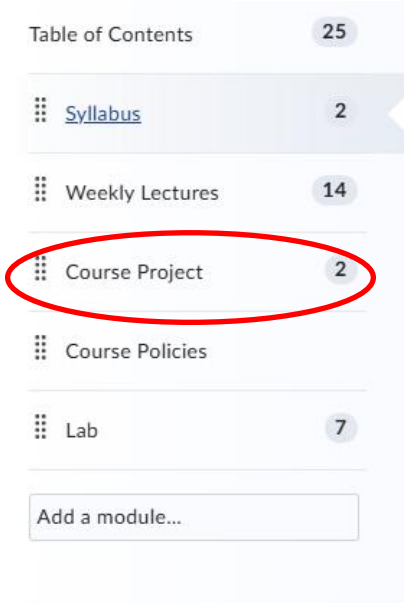
Challenges in DDBMS

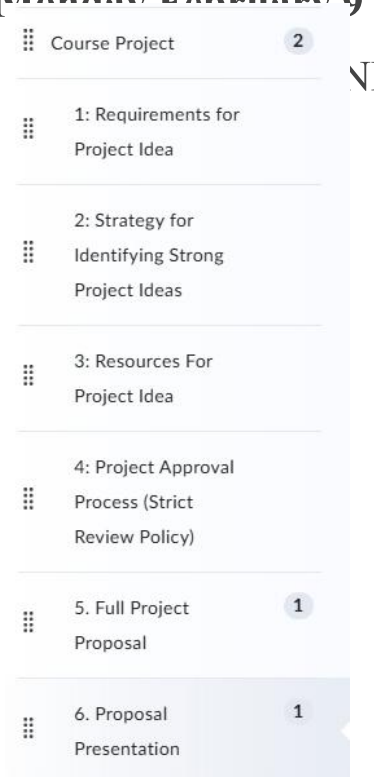
CAP Theorem

https://domains.upperlink.ng/elementor-947/

Announcement

- Proposal Presentation is on **Monday February 9 at the class**.
- Each presentation should be [illegible] ND Q&A for 2 to 3 minutes.





Procrastination! Is it healthy?

Motivation & Background

Distributed vs Centralized Databases

Multiple databases are distributed across different computers/nodes/servers .

vs

Entire database run on a single computer/node/server.

Google Spanner, Azure Cosmos DB, BigQuery, Data warehouses, etc.

vs

MySQL, SQL Server, PostgreSQL, etc.

Parallel vs Distributed Databases

a DBMS that runs across multiple processors and is designed to execute operations in parallel, whenever possible.

vs

a logically related collection of data that is shared which is physically distributed over a computer network on different sites.

Definition of a Distributed DBMS

A **Distributed Database Management System (DDBMS)** is a software system that manages a single logical database whose data is **physically distributed** across multiple sites (computers, servers, or data centers) connected by a network.

- ✓ To the user, it appears as **one integrated database system**.
- ✓ In reality, data may be fragmented, replicated, or partitioned across many machines.
- ✓ The DDBMS is responsible for:
  - **Transparency** – hiding the complexities of distribution.
  - **Coordination** – ensuring consistency, reliability, and query efficiency across sites.
  - **Autonomy** – allowing local sites to operate independently but still participate in the global system.

Why Distributed Databases?

1. **Growth of Data Across Geographies**

2. **Scalability and High Availability Needs**

3. **Limitations of Centralized DBMS**
   - ✓ **Single point of failure:** Outage = total service disruption.
   - ✓ **Bottlenecks:** A single server struggles under billions of requests.
   - ✓ **Geographical latency:** Users far away from the central server experience delays.
   - ✓ **Regulatory challenges:** Data locality laws (e.g., GDPR, HIPAA) may require storing data within certain regions ⟶ centralization becomes impractical.

Real-world examples:

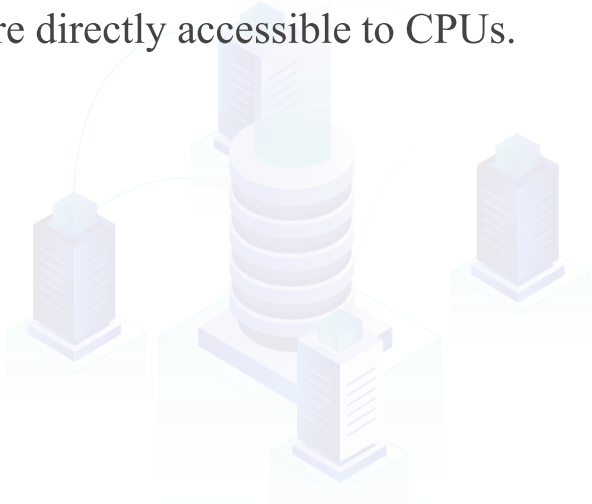
Google Cloud Spanner

Amazon Aurora

CockroachDB

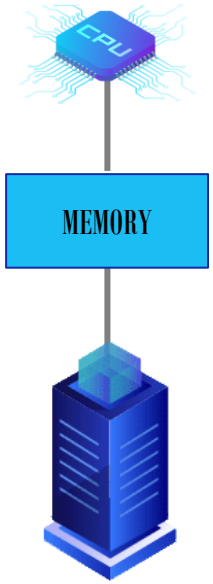
System Architectures

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.



System Architectures

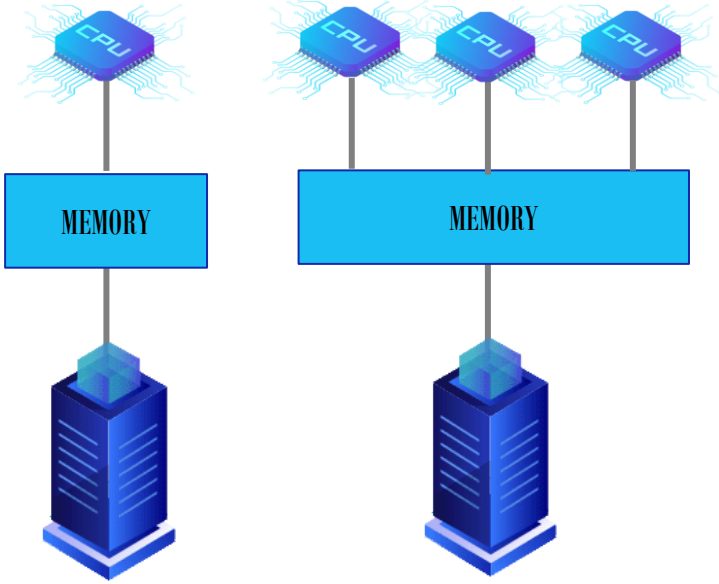
A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.



Shared Everything

System Architectures: Scaling to Higher Load

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.

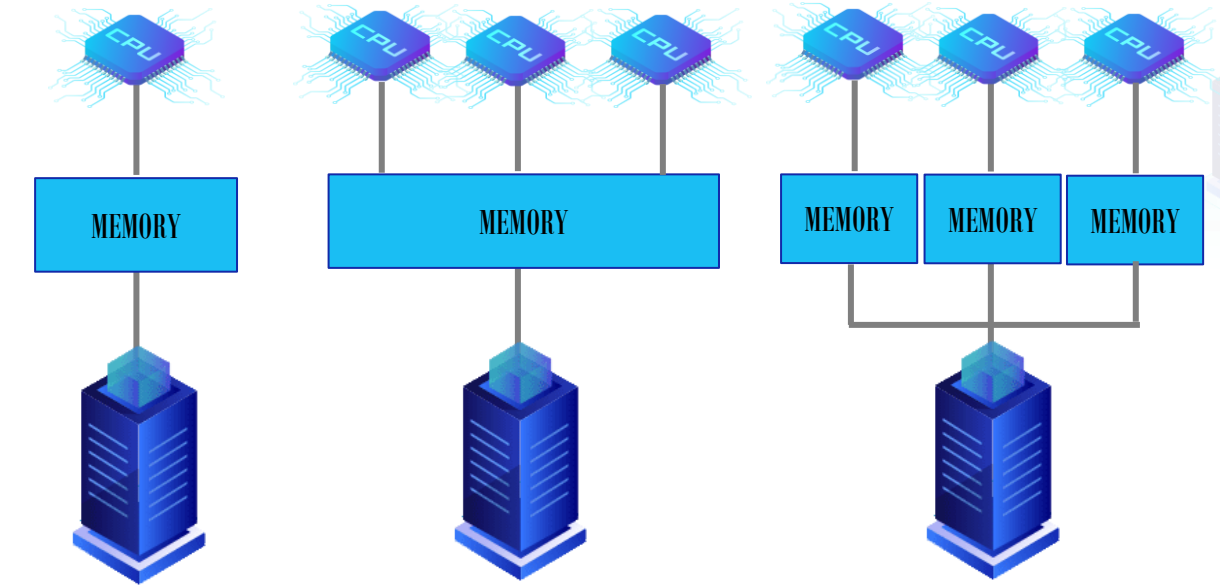


Shared Everything

Shared Memory

System Architectures: Scaling to Higher Load

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.



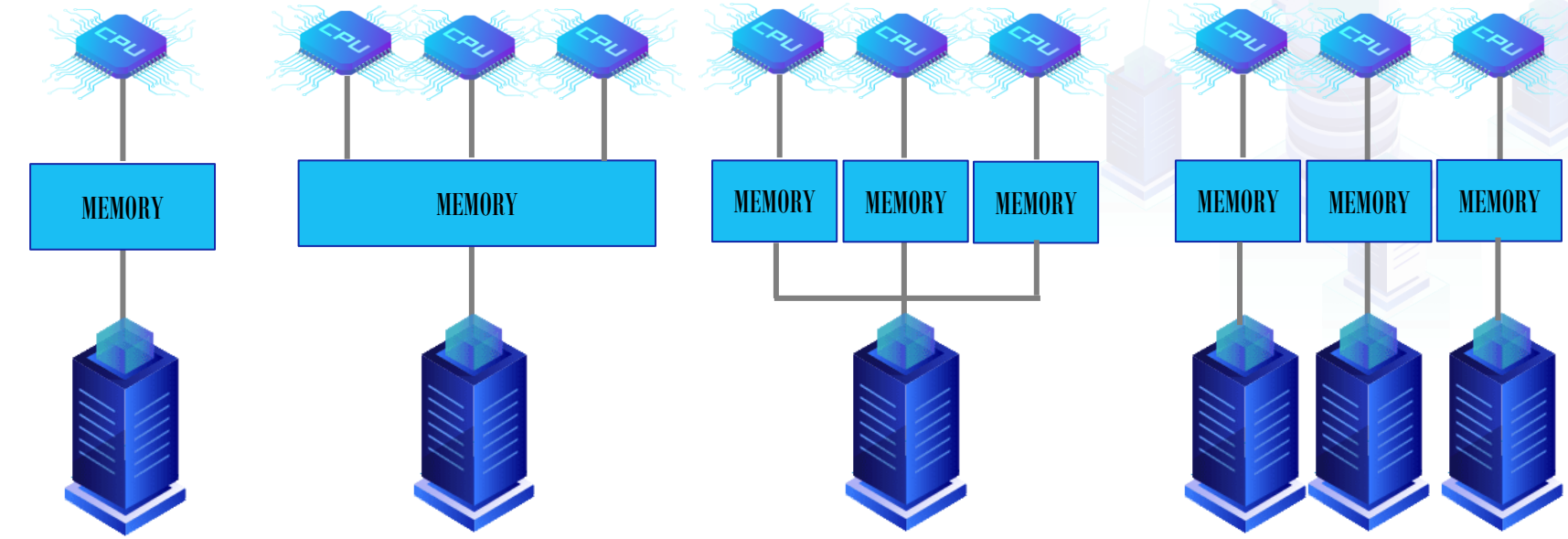
Shared Everything

Shared Memory

Shared Disk

System Architectures: Scaling to Higher Load

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.



Shared Everything

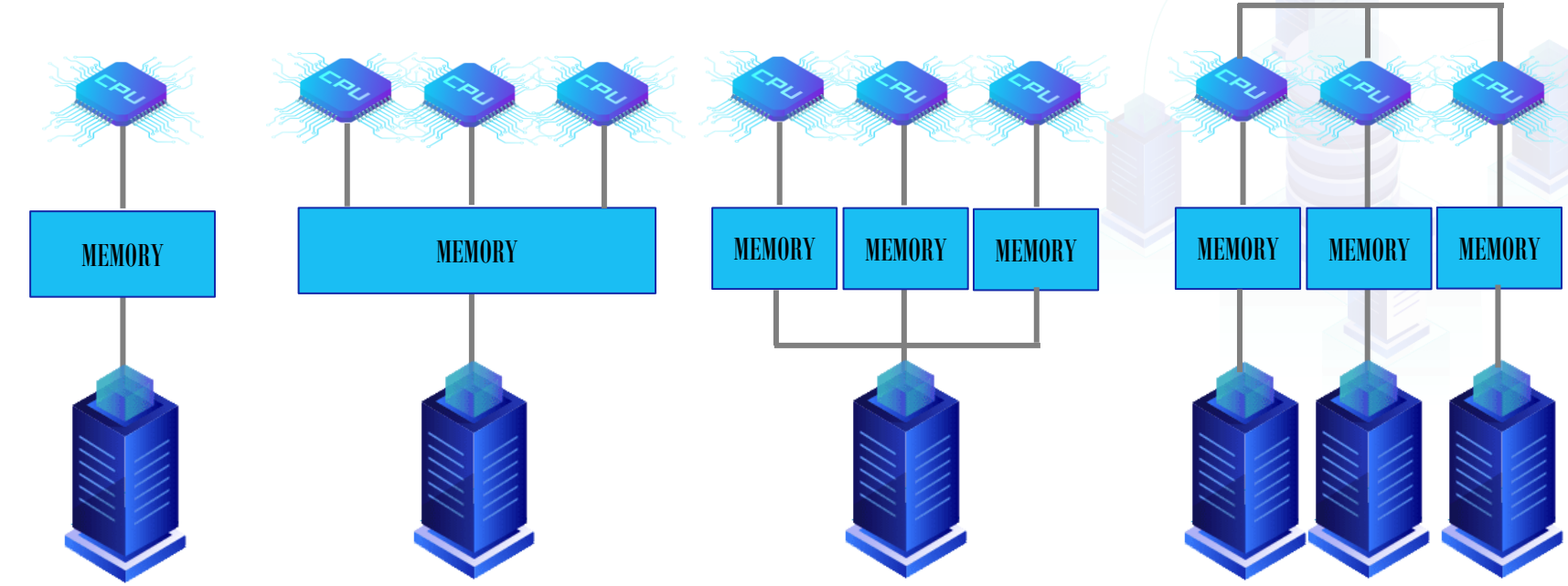
Shared Memory

Shared Disk

Shared Nothing

System Architectures: Scaling to Higher Load

A DBMS's system architecture specifies what shared resources are directly accessible to CPUs.



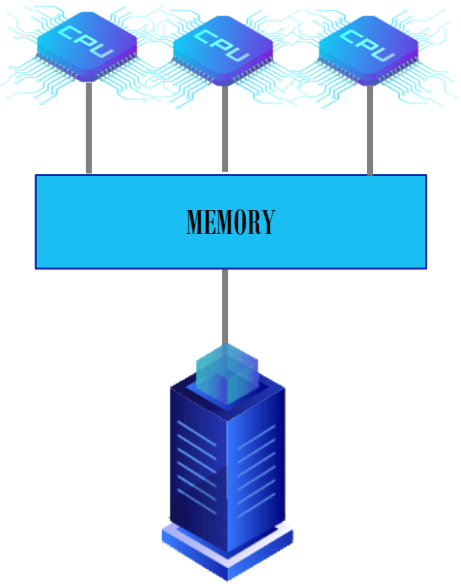
Shared Everything

Shared Memory

Shared Disk

Shared Nothing

Shared Memory



Shared Memory

Definition: All processors share the **same memory space** and have equal access to it. The database buffer pool and other structures reside in shared memory.

How it works:

- ✓ All the CPUs can access the same memory and are all controlled by a single operating system.
- ✓ A fast interconnection network allows any processor to access any part of the memory in parallel.

Advantages:

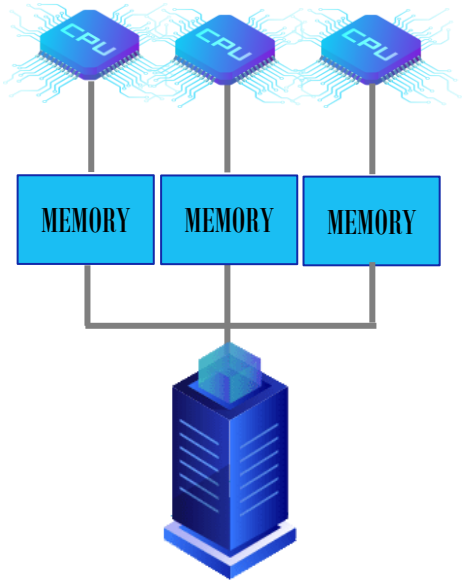
- ✓ Simple to program (looks like a single system).
- ✓ Very fast communication between processors (just reading/writing memory).

Disadvantages:

- ✓ Scalability is limited ⟶ as the number of processors increases, contention for memory access becomes a bottleneck.
- ✓ Hardware cost is high because shared memory requires special interconnects.

Use cases: Small-scale multiprocessor servers (e.g., early Oracle RAC on SMP systems).

Shared Disk



Shared Disk

Definition: Each processor has its **own private memory**, but all processors share the **same disk storage**.

How it works:

- ✓ Processors execute transactions independently.
- ✓ The shared disk ensures that every node can access all data.
- ✓ A global lock manager is often required to maintain concurrency.

Advantages:

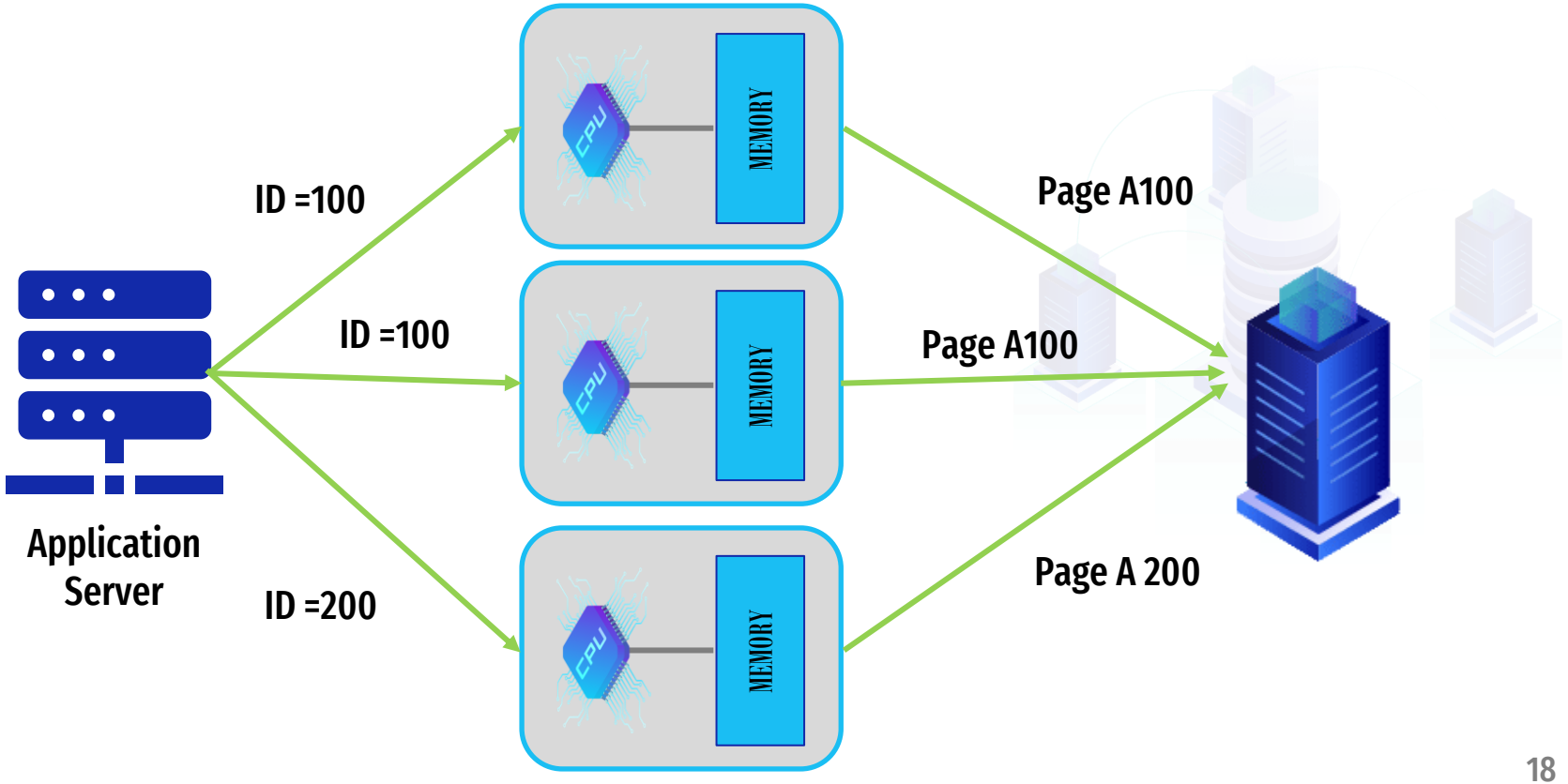
- ✓ High availability: if one node fails, others can still access the same disk.
- ✓ Easier load balancing than shared-memory because memory is independent.

Disadvantages:

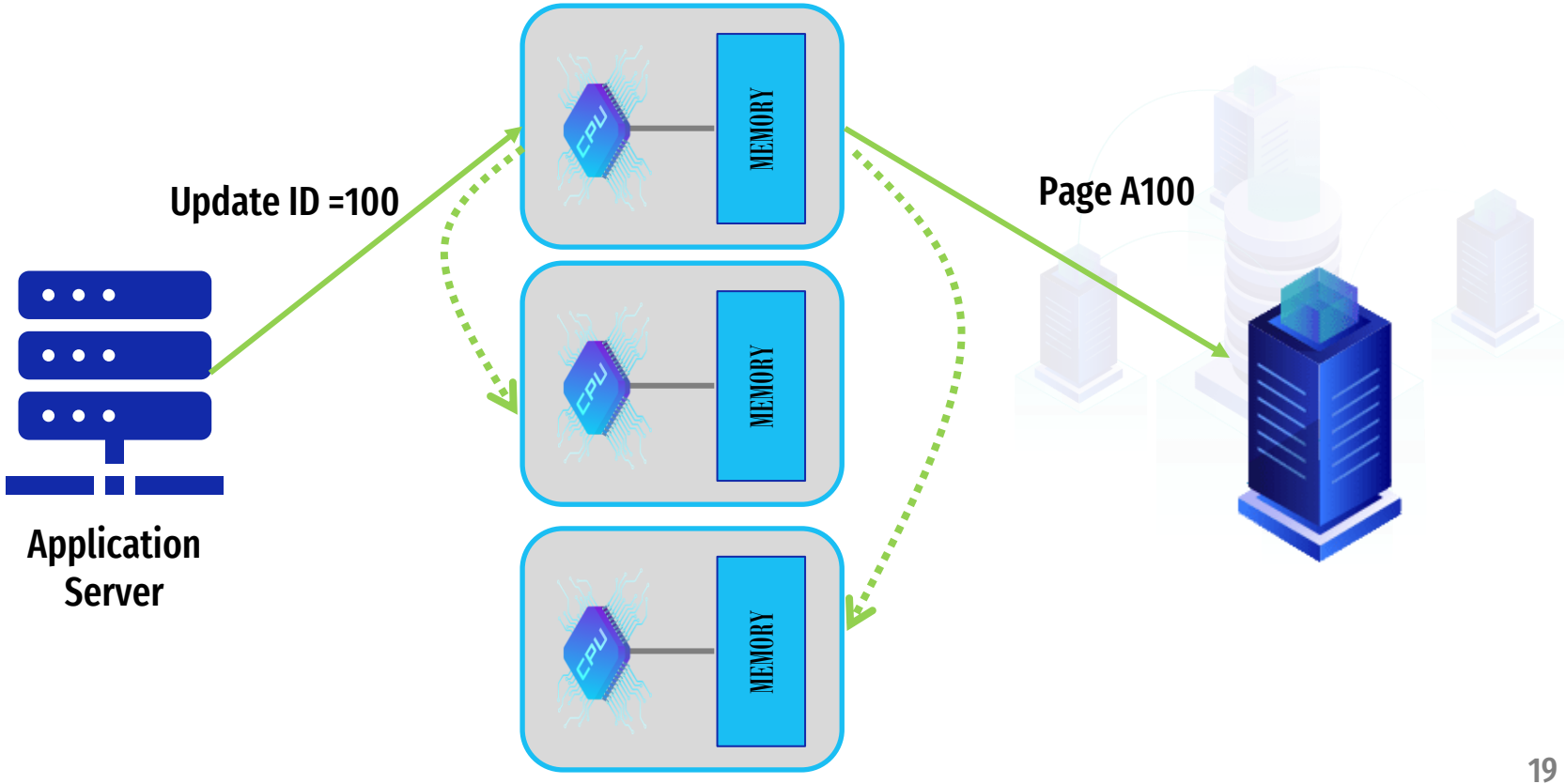
- ✓ Disk access can become a bottleneck if many processors contend for I/O.
- ✓ Requires careful coordination of caching and locking across nodes.

Use cases: Clustered databases like **Oracle RAC**, IBM DB2 PureScale.

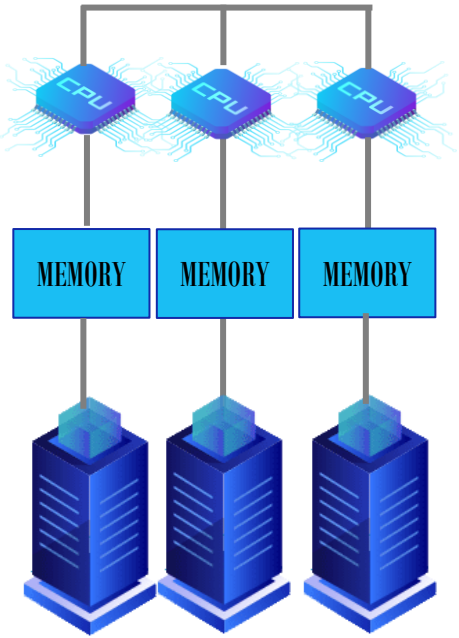
Shared Disk Example



Shared Disk Example



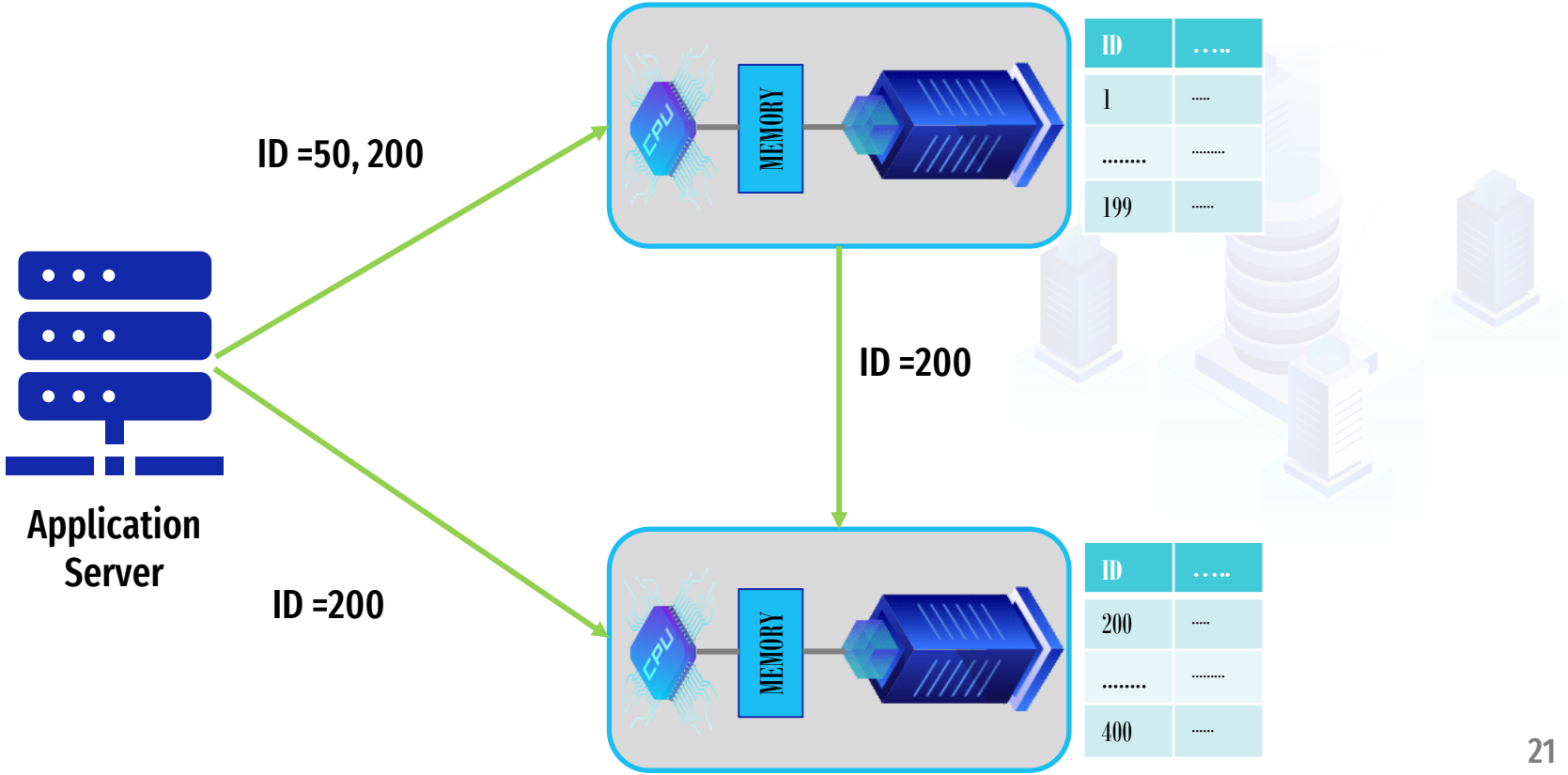
Shared Nothing



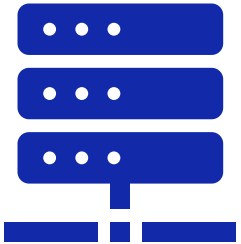
Shared Nothing

- **Definition**: Each node (processor) has **its own CPU, memory, and disk**. No resources are physically shared. Nodes communicate only by passing messages over a network.
- **How it works**:
  - ✓ Data is partitioned/sharded across nodes.
  - ✓ Each node is responsible for its subset of the database.
  - ✓ Queries that need multiple nodes' data involve **distributed query processing**.
- **Advantages**:
  - ✓ Highly scalable ⟶ just add more nodes.
  - ✓ Fault isolation: failure of one node affects only its own data.
  - ✓ Low contention since no resources are shared.
- **Disadvantages**:
  - ✓ More complex query processing and transaction management.
  - ✓ Communication cost between nodes can be high.
- **Use cases**: Modern large-scale distributed systems ⟶ **Google Spanner, CockroachDB, Amazon Aurora, Hadoop/Hive**.

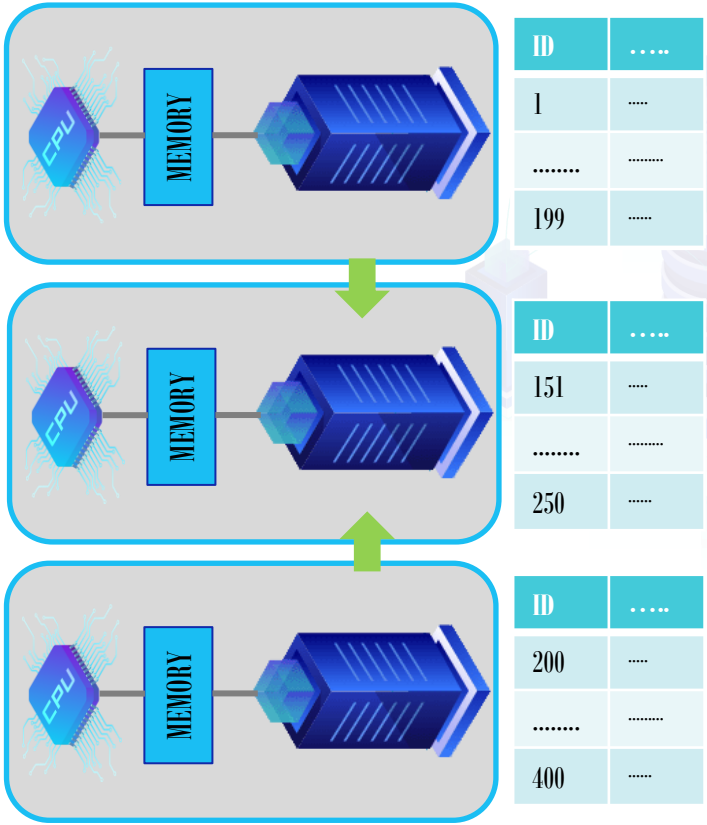
Shared Nothing Example



Shared Nothing Example



Application Server



Re-Define Distributed Databases

A distributed database management system (DDBMS) is the software that manages a collection of multiple, logically interrelated databases located at the nodes of a distributed system and provides an access mechanism that makes this distribution transparent to the users. A distributed DBMS is logically integrated but physically distributed.

Key Characteristics

1. **Logical Unity, Physical Distribution**
   - ✓ One database image; data stored in different places.
2. **Transparency (core principle)**
   - ✓ **Location Transparency**: users don't need to know where data resides.
   - ✓ **Replication Transparency**: multiple copies exist, but system ensures consistency.
   - ✓ **Fragmentation Transparency**: data can be partitioned across sites, but queries still work as if data were centralized.
3. **Distributed Query & Transaction Management**
   - ✓ Queries may need to access multiple sites.
   - ✓ Transactions must preserve **ACID** properties across the network.
4. **Reliability and Availability**
   - ✓ System must handle site failures and network partitions gracefully.
5. **Heterogeneity Support** (in some cases)
   - ✓ A DDBMS can integrate databases from different vendors or data models (relational, document, key-value).

Example of a Distributed DBMSs 1

"**Google** runs on hundreds of thousands of servers—by one estimate, in excess of 450,000—racked up in thousands of clusters in dozens of data centers around the world. It has data centers in Dublin, Ireland; in Virginia; and in California, where it just acquired the million-square-foot headquarters it had been leasing. Google runs more than 2,000,000 servers in

History of Distributed DBMS

Peer-to-peer systems (P2P)

Client/server

Cloud computing

- Infrastructure-as-a-service (IaaS)
- Platform-as-a-service (PaaS)
- Software as- a-service (SaaS)
- * Database-as-a-service (DBaaS)

Homogeneous and Heterogeneous DDBMSs

Homogenous Nodes

All sites use the same DBMS product

Much easier to design and manage.

Heterogenous Nodes

Sites may run different DBMS products.

Translations are required to allow communication between different DBMSs.

Data may be required from another site that may have:

- ✓ different hardware;
- ✓ different DBMS products;
- ✓ Both different hardware and different DBMS products.

Functions of a DDBMS

Use the same concept in single node DBMSs to support transaction processing and query execution in distributed environments.

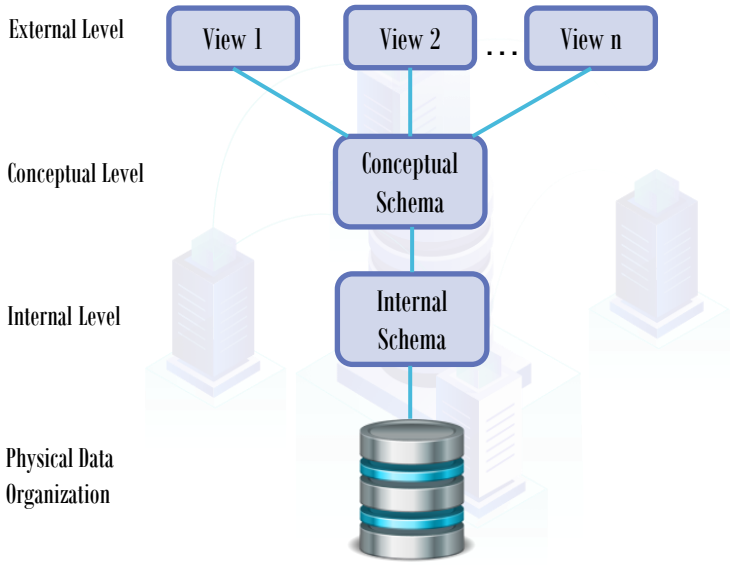
Query optimization & Execution, Concurrency Control, Logging & Recovery, Communication services & Security control

In addition, we expect a DDBMS to have the following functionality:

- ✓ Extend communication services to provide access to remote sites and allow to transfer of queries and data among the sites using a network;
- ✓ Extend system catalog to store data distribution details;
- ✓ Distribute query processing, including query optimization and remote data access;
- ✓ Extend security control to maintain appropriate authorization/access privileges to the distributed data;
- ✓ Extend concurrency control to maintain consistency of distributed and possibly replicated data;
- ✓ Extend recovery services to take account of failures of individual sites and the failures of communication links.

Architectures of a DDBMS

Due to diversity of distributed DBMSs, there is no accepted architecture equivalent to ANSI/SPARC 3-level architecture. However, it may be useful to present one possible reference architecture that addresses data distribution.



ANSI/SPARC 3-level architecture

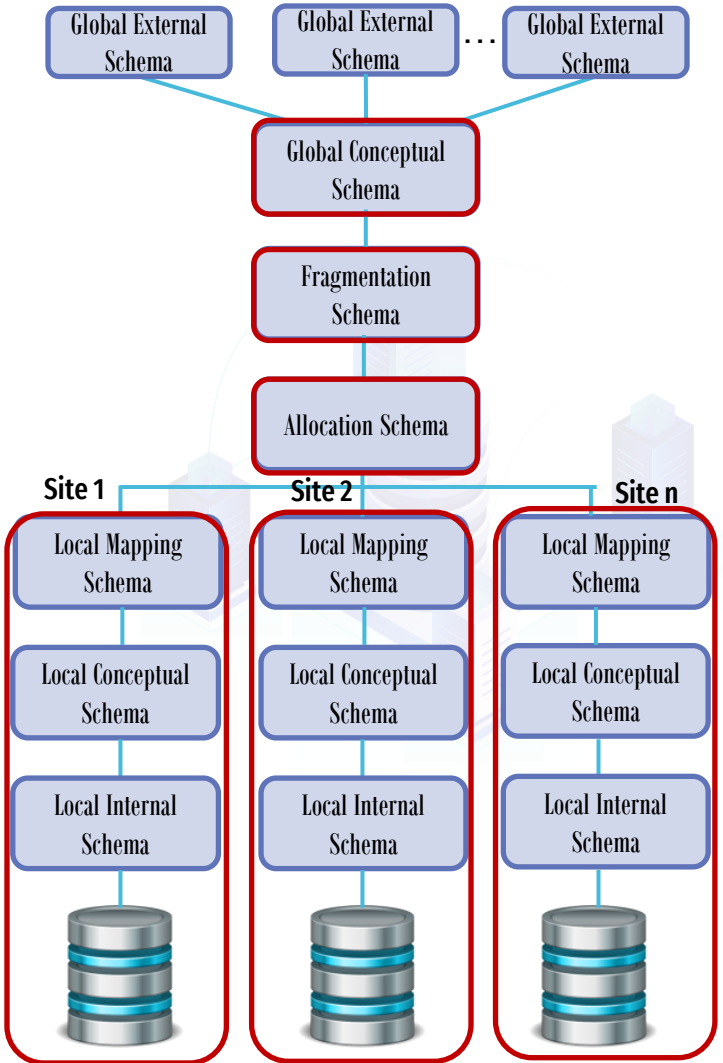
Architectures of a DDBMS

Due to diversity of distributed DBMSs, there is no accepted architecture equivalent to ANSI/SPARC 3-level architecture.

However, it may be useful to present one possible reference architecture that addresses data distribution.

Reference Architecture for a DDBMS consists of the following schemas:

- ✓ Set of global external schemas.
- ✓ Global conceptual schema (GCS).
- ✓ Fragmentation schema and allocation schema.
- ✓ Set of schemas for each local DBMS conforming to 3-level ANSI/SPARC.



Challenges in Distributed DBMS

1. Data Distribution and Partitioning

- ✓ Data is physically split across multiple nodes (shards/partitions) to improve scalability.
- ✓ **Challenges:**
  - How to **divide data efficiently**? Horizontal vs vertical partitioning, hybrid strategies.
  - Ensuring queries that span multiple nodes are **efficient and correct**.
  - Handling **data skew**, where some nodes get more load than others.

2. Data Replication and Consistency

- ✓ Data is often replicated for availability and fault tolerance.
- ✓ **Challenges:**
  - Keeping **all replicas consistent** during updates.
  - Choosing between **strong consistency** (all nodes see the same value) or **eventual consistency** (updates propagate asynchronously).

Challenges in Distributed DBMS

3. **Network Failures and Partitions**
   - ✓ Nodes communicate over networks that can fail or become partitioned.
   - ✓ **Challenges:**
     - Maintaining **system availability** when some nodes cannot communicate.
     - Handling **split-brain scenarios** where disconnected nodes make conflicting updates.

4. **Distributed Transactions and Concurrency**
   - ✓ Transactions may involve multiple nodes, each holding part of the data.
   - ✓ **Challenges:**
     - Ensuring **atomicity**: either all nodes commit or none do.
     - Maintaining **isolation**: preventing conflicts between concurrent transactions across nodes.
     - Coordinating **commit protocols** like 2PC/3PC or using Paxos/Raft for consensus.

Challenges in Distributed DBMS

5. Latency and Communication Overhead

- ✓ Data and transactions often require cross-node communication.
- ✓ **Challenges:**
  - Network latency is orders of magnitude higher than local memory access.
  - Query optimization must **minimize data movement** across nodes.

6. Fault Tolerance and Recovery

- ✓ Nodes can fail, disks can crash, networks can partition.
- ✓ **Challenges:**
  - Detecting failures quickly and **recovering without losing data**.
  - Handling **partial failures**, where some nodes fail but others remain operational.
  - Maintaining ACID properties and durability during recovery.

Challenges in Distributed DBMS

7. Security and Access Control

- ✓ Distributed systems often span multiple locations and organizations.
- ✓ **Challenges:**
  - Ensuring **secure communication** between nodes.
  - Enforcing **global access policies** consistently.
  - Handling **regulatory requirements** (GDPR, HIPAA) for data storage location.

CAP Theorem

In distributed databases, nodes can fail, or the network can partition. Designers must make trade-offs between three properties: Consistency, Availability, and Partition Tolerance.

The CAP theorem, also known as **Brewer's theorem**, states that any distributed data store can only guarantee two out of three desirable properties:

C – Consistency: Every user sees the same data at the same time.

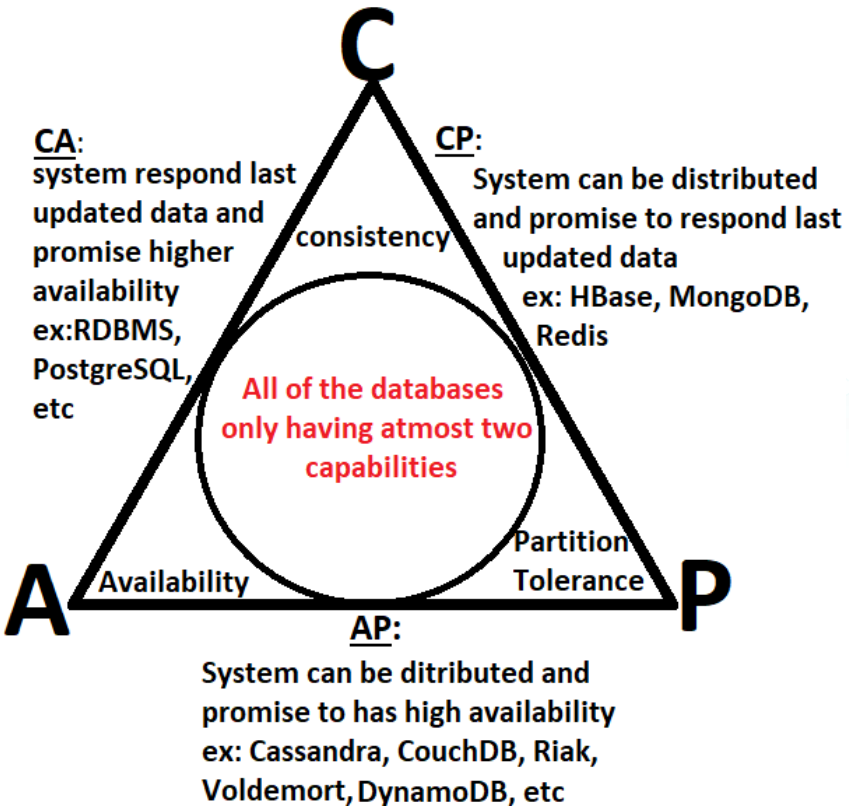
A – Availability: The system always responds, even if some parts fail.

P – Partition Tolerance: The system keeps working even if network links break between servers.

CP = correctness first

AP = availability first

CA = only possible in non-distributed systems



Source: https://www.geeksforgeeks.org/dbms/the-cap-theorem-in-dbms/

The 3 Trade-off Categories

1. **CP Systems (Consistency + Partition Tolerance)**
   - ✓ Always keep data consistent, even if it means sometimes rejecting requests.
   - ✓ **Availability may suffer** during network issues.
   - ✓ **Example (General):** A **banking system** that ensures your balance is *always correct*. If servers can't talk to each other, it may block transactions instead of risking inconsistency.
2. **AP Systems (Availability + Partition Tolerance)**
   - ✓ Always stay available, even if data might be temporarily inconsistent.
   - ✓ **Consistency is eventual** (things will line up later).
   - ✓ **Example (General): A shopping cart system** in e-commerce. Even if servers are split, users can still add items to cart. Later, the system reconciles differences.
3. **CA Systems (Consistency + Availability)**
   - ✓ In theory, strong consistency + full availability.
   - ✓ But in a *distributed* world, when partitions happen, CA is impossible.
   - ✓ **Example (General, single server):** A **standalone relational database** (like Oracle running on one machine). As long as the server doesn't split across networks, it gives both consistency and availability.

Connect CAP to Architectures

Architecture	Typical CAP Focus	Why
Shared-Memory	**C** + **A**, P irrelevant locally	All nodes share memory; no network partitions internally. Scaling is limited.
Shared-Disk	**C** + **A** (with cluster coordination)	Nodes share disk, global lock manager ensures consistency; network partitions can break availability.
Shared-Nothing	Trade-offs explicit **C** + **P** or **A** + **P**	Data is fully distributed; partitioning is inevitable; system must choose between strong consistency or high availability during partitions.

Map Real-World Systems to CAP

- **Google Spanner ⟶ C + P**
  - ✓ Prioritizes **strong consistency** globally, even if network partitions slightly delay availability.
  - ✓ Uses **TrueTime** + **Paxos** to maintain global consistency.
- **Cassandra ⟶ A + P**
  - ✓ Prioritizes **availability and partition tolerance**.
  - ✓ Accepts **eventual consistency**, meaning some reads might be stale temporarily.
- **CockroachDB ⟶ C + P**
  - ✓ Spanner-inspired, emphasizes strong consistency with distributed consensus (Raft).

Summary

Major different between DDBMS and Centralized DBMS.

Various type of system architectures: Shared Everything, Shared Memory, Shared Disk and Shared Nothing

Define DDBMS: the software that manages a collection of multiple, logically interrelated databases located at the nodes of a distributed system.

Major Advantages and Disadvantages of DDBMSs.

Homogeneous and Heterogeneous DDBMSs.

Owing to the diversity of distributed DBMSs, it is much more difficult to present an equivalent architecture that is generally applicable.